

Chapter 4

Procedures

So far you have been writing code in event procedures. It is fine since the programs are not long. As your programs become more and more complicated and code grows to hundreds or thousands of lines, writing all code in one event procedure would become difficult and sometimes impossible.

A procedure is a self-contained unit of code made up of a series of computational steps to carry out certain tasks. It can be called by other procedures including events in the same project as it is needed. Procedures are commonly used in programming to decompose large and complex jobs into modules. For example, overlay analysis in GIS is a complex job. To write a program to overlay two polygon feature classes, we can first break the job down into three major modules: 1) read polygon features – to read all polygon features from the input feature classes (e.g. shapefiles on the hard drive) into the memory; 2) overlay features – this is still a quite large module that needs to be further decomposed; 3) save results – to resulting features in an output shapefile. We can continue to break the second module down into three smaller modules: intersecting arcs, assembling new polygons, and link attribute data to new polygons (Figure 1). If necessary, any of these modules can be further decomposed into even smaller modules. After decomposing the large job into small and manageable modules, each can be implemented by a procedure.

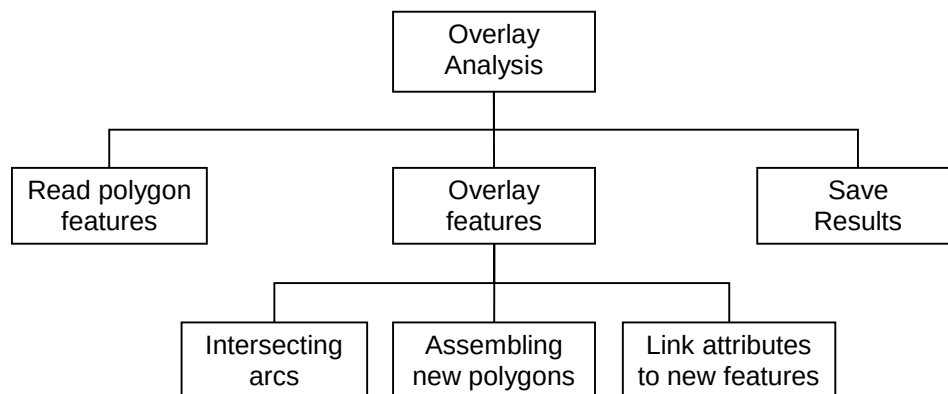


Figure 1 Breaking down a complex job into modules

Modular programming is a software design technique that emphasizes separating the functionality of a program into independent, interchangeable modules each carrying out a specific task. Modular programming can facilitate project management and collaboration since independently functional modules can be implemented by different programmers. Once all modules are implemented as procedures, they can be assembled to make a product. Moreover, the independently functional procedures are reusable, which can also enhance software productivity. For example, if a procedure that creates new shapefiles is created in one project, it may be called as many times as necessary to create shapefiles, and it can be easily copied into other projects when necessary.

This chapter introduces two types of procedures supported by Visual Basic, *Function* and *Sub procedure* (also called *subroutine*, or simply *sub*). It will cover the structure of these

procedures, procedure calling, scope of variables, passing arguments to parameters by value and by reference.

1. Function

A function is a procedure that usually takes some input data, processes them, and returns one output. For example, the built-in functions CInt() takes a string consisting of numeric characters as input and returns an integer number.

```
Dim n As Integer = CInt("10")
```

Where string "10" is an input. Except for a string constant, an input to the CInt() function can also be a string variable carrying a value. The function processes the input data ("10") and returns an integer value (10) as result which is immediately assigned to a new integer variable *n* in the above example.

Other built-in functions you have used so far include the CStr() function which converts a number into a string (e.g. to convert number 3.14 to string "3.14"), the Asc() function which converts a character into the ASCII code, and the Chr() function which converts an ASCII code into a character. You may name a few more built-in functions.

You can create your own functions in VB. For example, you may create a function that takes in a temperature in Fahrenheit and converts it into Celsius temperature. You may also create a function that takes in two numbers and returns the average of them. The input data required by a function are called *parameters*. In VB, the general form of a function is:

```
Private Function FunctionName([parameter list]) As <data type>
    ' computation ...
    Return (result)
End Function
```

In the header line, keyword Private (Public or other) defines the scope of the function, which will be discussed in detail in Chapter 6. At this point, you can safely use keyword Private. Keyword Function declares that this procedure is a function instead of a sub procedure. Following the Function keyword is a function name. A function name is usually a word or a combination of words that describe the purpose of the function or the task the function performs. To create a temperature conversion function, for example, you may name it *FtoC* (i.e. Fahrenheit to Celsius) or *CtoF* (Celsius to Fahrenheit). To create a function that computes the average of two numbers, you may name the function *Average*. Like variable names, a function name should not contain spaces and certain special characters such as slash ("/"), backward slash ("\"), dot symbol, math operator symbols, parentheses, etc. Underscore symbol ("_") is allowed in function names and is often used as a substitute for space. A function name should not coincide with any of the VB reserved keywords (Please refer to <http://msdn.microsoft.com/en-us/library/dd409611.aspx>).

Following the function name is a pair of round parentheses containing a list of parameters. Parameters are variables used to receive data provided by the client. In the parameter list, each parameter must be declared with a data type but without keyword Dim, e.g.

temperature As Single. Multiple parameters in the list are separated by commas. When a client procedure calls a function, the client must provide an exact number and type of values that match the list of parameters of the function, so that each parameter of the function can receive a valid input value for the function to process. You are recommended to use succinct and informative names for parameters so that when you write code to call the procedure in a client procedure, IntelliSense can display an informative parameter list.



Before VB 2010 Service Pack 1 (SP1) was released in March 2011, keyword `ByVal` is automatically added ahead of each parameter in the parameter list if omitted when you program. This behavior was lifted by VB 2010 SP1. Keywords `ByVal` and `ByRef` will be discussed in Section 5.

The last component of the header line of a function is a declaration of the return data type of the function. Inside a function block is a list of instructions that carry out the tasks, and the result of computation is returned to the client by keyword `Return`. Finally, a function is closed by statement `End Function`.



Example 1: Create a new VB 2010 project named “Temperature” with one button named `btnConvert` on the main form. To create a function that converts temperatures from Fahrenheit into Celsius, enter the following function inside the `Form1` class block in the code editor.

Function name Declares a parameter Specifies the return data type

```
Private Function FtoC(tf As Integer) As Double
    ' computation
    Dim tc As Double
    tc = (tf - 32) * 5 / 9
    Return tc      ' returns the result
End Function
```

Function `FtoC` is declared with one Integer parameter `tf` and a Double return data type. When a client calls this function, it must provide an Integer input value which will be received by parameter `tf`. The return data type (Double) specifies that this function will produce a result of Double data type. Inside the function, parameter `tf` is computed and the result of computation is assigned to variable `tc` which is declared as Double. Finally, statement `Return tc` sends the result (`tc`) back to the client. Therefore, the data type of variable `tc` must match the return data type declared in the function header. Similarly, we can create a function to calculate the average of two numbers as the following:

```
Private Function Average(a As Integer, b As Integer) As Double
    Return (a + b) / 2      ' computation result is directly returned
End Function
```

Function `Average` is declared with two integer parameters (`a As Integer, b As Integer`). This means that this function requires two integer numbers as input when called. Depending on needs of the task, parameters can be declared as different data types. This

Average function is declared with a Double return data type, which means that the function will produce a Double precision floating point number as result when executed. Can we declare the return data type as Integer? No, because when an integer number is divided by another integer number in VB, the result is a Double precision floating point number.

Once you create a function, you can call it in other procedures, such as in a button's Click event procedure, just like you call a built-in function. To complete Example 1, please call the *FtoC* function in the btnConvert button's click event procedure as the following code.



```
Private Sub btnConvert_Click(...) Handles btnConvert.Click
    Dim f As Integer = 70          ' temperature in Fahrenheit
    Dim c As Double = FtoC(f)      ' calls function FtoC(), passing value f to it
    MessageBox.Show(f & Chr(176) & "F = " &
        Math.Round(c, 1) & Chr(176) & "C")

    c = FtoC(100)                 ' calls FtoC() again, passing constant value 100 to it
    MessageBox.Show("100" & Chr(176) & "F = " &
        Math.Round(c, 1) & Chr(176) & "C")
End Sub
```

Function *FtoC()* is called twice in the button's Click event procedure above. Since the function has an Integer parameter, you must provide an integer value to it as input when you call it. A value passed to a parameter of a function is called an *argument*. In the first call, variable *f* which carries integer value 70 is sent to the function. In the second call a constant integer number 100 is directly passed to the function. In each, the result returned from the function is assigned to variable *c*. A message box is used to display the value of variable *c* after calling function *FtoC()* (Figure 2). To display a degree symbol (°) which does not exist on a standard keyboard, the program calls a built-in function *Chr()* to convert ASCII code 176 to the degree character.

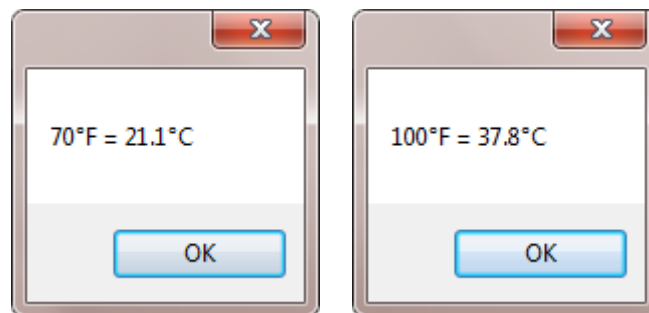


Figure 2 Running results of temperature conversions



If you want to create a temperature lookup table, you can use a For ... Next loop to call the *FtoC()* function to compute a range of values and display results in a list box. Please try it yourself.

To test the *Average()* function, you can create a button (*btnTest*) on the main form, then implement the Click event procedure as the following:

```
Private Sub btnTest_Click(...) Handles btnTest.Click
    Dim x As Integer = 10
```

```

Dim y As Integer = 15
Dim avg As Double = Average(x, y)      ' calls the Average() function
MessageBox.Show("Average of " & x & " and " & y & " is " & avg)
End Sub

```

The Average() function is called in the third line inside the button event procedure. Because the function has two parameters thus requires two integer values as input, the client code, i.e. the line that calls the function, passes two arguments to the function: variable x carrying value 10 and variable y carrying value 15.



Important:

- To call a function, the client procedure must provide arguments that match the parameters of the function in terms of number, type, as well as order.
- Values passed to a function can be constants (e.g. 100), variables that carry definite values, or expressions that can result in definite values, e.g. x^2+5 , in which x has been assigned a value.
- A function's return data type must be declared to match the data type of the computation result to be returned by a function. For example, the Average() function is declared with a Double return data type which matches the data type of the computation result $((\text{num1} + \text{num2}) / 2)$.
- The order of procedures in a class does not matter, but procedures cannot be nested or chained. For example, the following are wrong use of functions:

Wrong: function nesting

```

Private Function A(...) As String
    ' some code ...
    Private Function B (...) As Integer
        ' code for function B ...
    End Function
    ' code ...
End Function

```

Wrong: function chaining

```

Private Function A(...) As Double
    ' code for function A ...

    Private Function B(...) As Single
        ' code for function B ...
    End Function

    Private Function C(...) As Integer
        ' code for function C ...
    End Function
    ' code ...
End Function

```

Example 2: Function `EarthDistance()` (code in file “earthdist.vb” in folder `Source_Code` on the CD-ROM) computes the distance between two locations on the Earth’s surface by taking into account the curvature of the ellipsoidal surface. Write a program to compute distances on the Earth’s surface by calling this function.



Create a new VB project named “Distance” and design the project GUI by referring to Figure 3. Open file “earthdist.vb” in the VB editor and copy the `EarthDistance` function into the form class, then implement the Click event procedure of the compute button.

Control	Name
Form	frmDistance
TextBox	txtLon1
TextBox	txtLat1
TextBox	txtLon2
TextBox	txtLat2
Button	btnCompute
TextBox	txtOutput

Figure 3 Project GUI of Example 2

```

Private Sub btnCompute_Click(...) Handles btnCompute.Click
    ' checks if user input is complete
    Dim inputComplete As Boolean = ValidateInput()
    If Not inputComplete Then Return

    ' reads a value from a text box, e.g. txtLon1.Text
    ' converts it from String to Double by using function CDb1(),
    ' and assigns it to a new variable
    Dim x1 As Double = CDb1(txtLon1.Text)
    Dim y1 As Double = CDb1(txtLat1.Text)
    Dim x2 As Double = CDb1(txtLon2.Text)
    Dim y2 As Double = CDb1(txtLat2.Text)

    ' calls the EarthDistance function to calculate distance
    Dim dist As Double = EarthDistance(x1, y1, x2, y2)
    txtOutput.Text = Math.Round(dist, 2) & "kilometers"
End Sub

Private Function EarthDistance(lon1 As Double, lat1 As Double,
                               lon2 As Double, lat2 As Double) As Double
    ' computing distance code ...
End Function

Private Function ValidateInput() As Boolean
    If txtLon1.Text = "" Then Return False
    If txtLat1.Text = "" Then Return False

```

Passing arguments to parameters

```

    If txtLon2.Text = "" Then Return False
    If txtLat2.Text = "" Then Return False
    Return True
End Function

```

Once the button click event procedure is started, the first line of code in the event calls function `ValidateInput()` to check if the user input is complete. The `ValidateInput()` function does not require any parameter. It checks each coordinate text box to make sure all boxes are not blank. If a text box is blank, i.e. the `Text` property is equal to a pair of blank quotes, the function immediately returns a Boolean value `False`. If all text boxes pass the check, the function returns a `True` value in the last line.

The result of function `ValidateInput()` is received by a Boolean variable *inputComplete*. The second line checks the value of variable *inputComplete*. If the value `False`, then `Not inputComplete` is true and statement `Return` is executed which causes the program to exit the event procedure. Otherwise, if variable *complete* carries value `True`, then `Not inputComplete` is false, statement `Return` is not executed, and the program will continue to execute the rest of the code in the procedure.

Following the input validation, the program uses four lines to read four user-input coordinates from the text boxes. In each line, the `Text` property of a text box is converted into a `Double` type number which is assigned to a new `Double` type variable declared inline. The fifth line in the event procedure calls the `EarthDistance()` function to calculate distance and stores the result in a new variable named *dist*. To call the function, the client code passes four `Double`-type arguments, `x1`, `y1`, `x2`, `y2`, each carrying a definite value, to the function because the `EarthDistance()` function declares four `Double` type parameters, `lon1`, `lat1`, `lon2`, and `lat2` all as `Double` type. Consequently, the values of the four arguments `x1`, `y1`, `x2`, and `y2` in the client procedure are received by parameters `lon1`, `lat1`, `lon2`, and `lat2` of the function respectively. In the diagram above, the red arrows show how arguments are passed from the client to parameters of the function.

2. Sub procedure

Like functions, a sub procedure is also a block of code that independently carries out some tasks. The main difference between them is that a sub procedure does not return an output to the client. Since a sub procedure does not return an output, the header line of a sub procedure does not declare a return data type like a function does. A general format of sub procedure is as the following:

```

Private Sub ProcedureName([parameter list])
    ' code that carries out tasks ...
End Sub

```

Depending on the needs of tasks, a sub procedure can also declare a list of parameters to receive input from the client. Like function parameters, parameters of a sub procedure are declared in a pair of parentheses following the procedure name, each parameter must be declared with a data type, and multiple parameters are separated by commas.

A sub procedures may not have parameters if it knows where to obtain data. For example, a sub procedure that reads the content of a file on the hard drive may not need to declare arguments if the file has a fixed path and name.

```
Private Sub ReadFile()  
    ' code that open and reads the file ...  
End Sub
```

3. Program flow in procedure calls

Inside a procedure, the program naturally flows from top to bottom if there are no branches or loops. If a code line calls another procedure, the program will flow out into the external procedure. After executing the external procedure, the program flow return to the calling line in the current procedure. Please follow the arrows labelled with order numbers in Figure 4 to analyze the program flow.

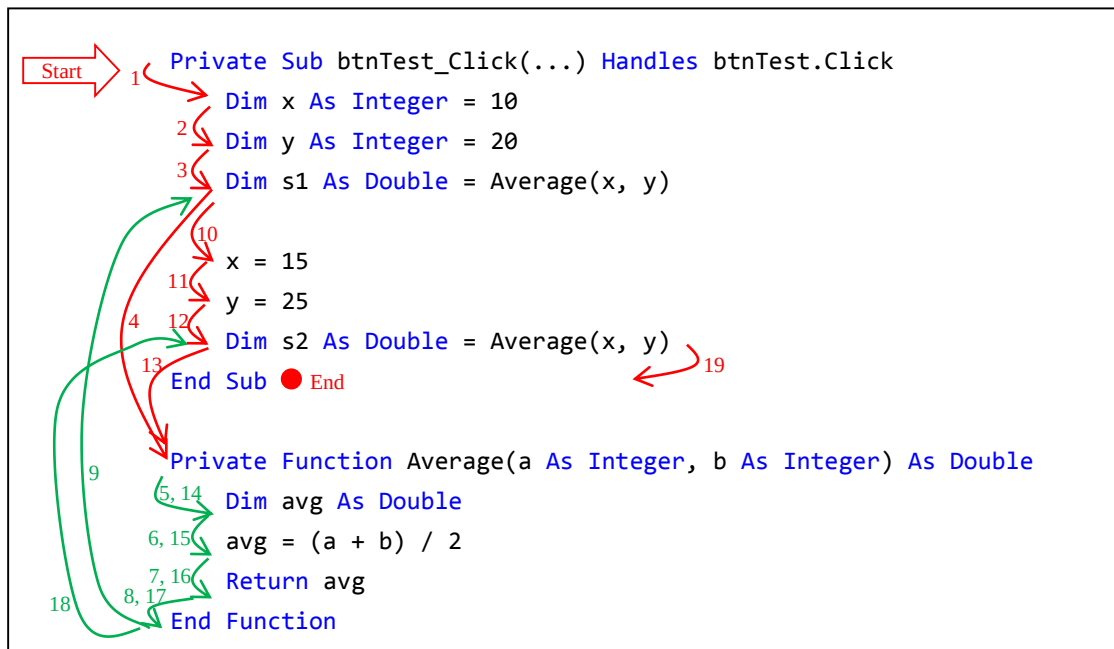


Figure 4 Program flows between procedures

Figure 4 shows that the Click event procedure of button `btnTest` calls the `Average()` function twice. So the button click event procedure is a client. The red arrows and numbers indicate program flow directions and step order. To watch the program flow, you can use the debug skills you learned in Chapter 4 to step through the program: set a breakpoint at the header of the button Click event procedure and start the program. Once the program flow stops at the breakpoint, use the F8 key to step through the program.



Example 3: Create a new VB project that uses a sub procedure to compute and display annual population growth rates between 2000 and 2010 for two cities given the following census data.

City	Year 2000	Year 2010
Phoenix	1,321,045	1,445,632
Flagstaff	52,904	65,870

Refer to Figure 5 to add a button named btnShow and a list box named lstResults in the main form. Then create a sub procedure Compute() and implement the button's Click event as the following.

```
Private Sub btnShow_Click(...) Handles btnShow.Click
    ' clear the list box
    lstResults.Items.Clear()

    ' declare some variables for use
    Dim city As String, pop00 As Long, pop10 As Long

    ' assign values to variables
    city = "Phoenix"
    pop00 = 1321045
    pop10 = 1445632

    Compute(city, pop00, pop10) ' calls the sub to compute and display result

    ' assign new values to the variables
    city = "Flagstaff"
    pop00 = 52904
    pop10 = 65870

    Compute(city, pop00, pop10) ' call the sub again
End Sub

Private Sub Compute(city As String, pop2000 As Integer, pop2010 As Integer)
    ' compute population growth rate
    Dim rate As Double = Math.Pow(pop2010 / pop2000, 1 / 10) - 1

    ' display result
    lstResults.Items.Add("Population growth rate of " & city & " is: " &
        Math.Round(rate * 100, 2) & "%")
End Sub
```

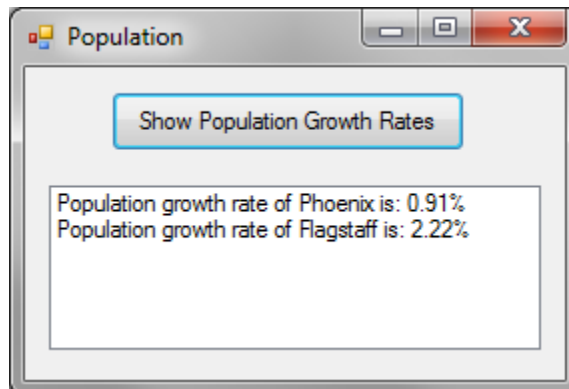


Figure 5 Execution result of Example 3

Use the VB debugging tools to find how the program flows with procedure calls. To do so, set a breakpoint at the header line of the button click event procedure and then start the program. Once the program flow stops, repeatedly use the shortcut key F8 to move a step at a time until all the code lines are executed.

4. Scope of variables

You have learned how to declare variables and procedure parameters (which are also variables but declared in procedure header lines without using the Dim keyword). Each variable has an effective scope and life span in which the variable is available (Figure 6). The following are a few rules regarding variable scopes:

- 1) Variables (including parameters) declared in a procedure are only available to that procedure. Once the procedure finishes an execution, i.e. when the closing statement End Sub or End Function is executed, all variables declared in it are deleted and the memory space allocated to these variables is released. One exception to this rule is variables declared with keyword Static instead of Dim. A static variable declared in a procedure is not deleted once the procedure finishes an execution. The value of a static variable can be carried into the next execution of the procedure.
- 2) Variables declared in a class are called *class-level variables*. Instead of using keyword Dim, class-level variables are usually declared with scope keyword Private or Public which are discussed in detail in Chapter 6. Class-level variables are available to all procedures in the same class. These variables are deleted only when the object instantiated from the class is deleted. In the case of a form class, class variables are deleted when the form object is destroyed which occurs when the application is closed.
- 3) Variables declared inside an If block or a repetition block are only available to that block. If the control variable of a For ... Next repetition is declared in the repetition opening line, it is only available to that loop block also.
- 4) A public variable declared in a class is available to ... It is too early to discuss this, will be introduced in Lecture 6.



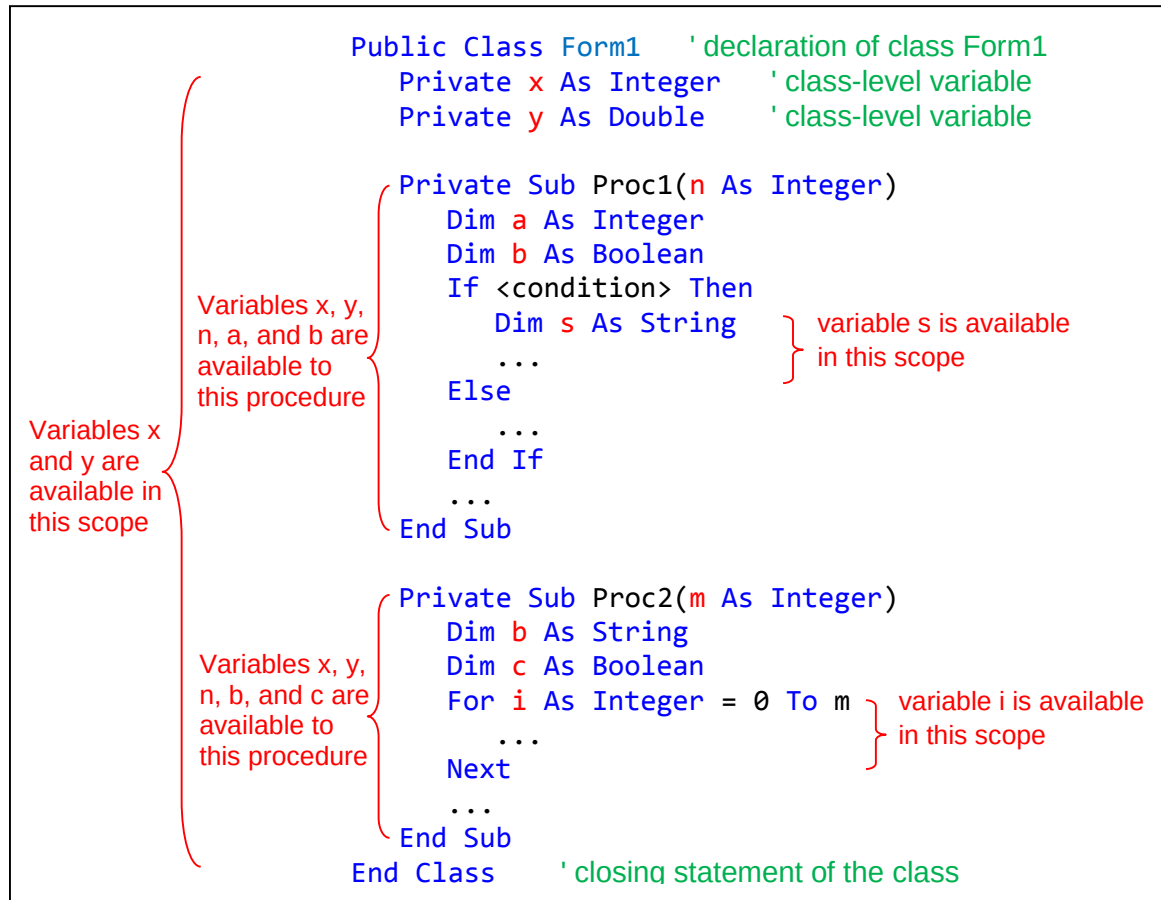


Figure 6 scopes of variables

In Figure 6, class Form1 contains two sub procedures, Proc1 and Proc2, and two private variables *x*, and *y*. Variables *x* and *y* are class-level variables, so they are available to both procedures, i.e. they can be used by code inside these procedures. Procedure Proc1 declares one parameter *n* and two internal variables *a* and *b*. Therefore, variables *n*, *a*, and *b* are local to this procedure. When Proc1 is called, these variables (*n*, *a*, and *b*) will be sequentially created. After execution of the procedure, these variables are deleted. If procedure Proc1 is called again, these variables will be created and deleted again. Procedure Proc1 contains an If block. If the condition turns out to be true while the procedure is being executed, variable *s* will be created in the sub-block under the true condition. Variable *s* is by no means available to code in the Else sub-block.

The same rules apply to variables declared in procedure Proc2. Proc2 contains a For ... Next loop. In this repetition block, control variable *i* is declared in the loop declaration line, therefore, the variable is only available to this repetition block.

Procedure Proc1 does not see variables of Proc2, and Proc2 does not see variables of Proc1 either. Although both sub procedures declare variable named “b”, but these are two different variables, i.e. different memory spaces, and they do not bother each other.

As you can see, procedures are quite independent and insulated from each other. Variables declared by a procedure are confined in that procedure. However, one

procedure can communicate with another procedure by calling it and passing arguments to it. For example, sub Proc1 may call sub Proc2 and pass an argument to Proc2. You have seen how arguments are passed from a client procedure to a procedure being called in previous sections of this chapter.

5. Passing Arguments ByVal vs. ByRef

5.1 ByVal

Parameters declared in the header line of a procedure are same as variables declared inside the procedure – a piece of memory space set aside to hold values of a specified data type. When a procedure is called, its parameters are responsible for receiving incoming data (arguments) provided by the client. In this process, incoming data are usually copied into the parameter memory spaces (Figure 7). Copying argument values to parameters is the default method of VB – a process called passing arguments by value (ByVal). Keyword ByVal is traditionally required in parameter declaration in the procedure header to indicate the by-value argument passing method. Since VB 2010 SP1, it has become optional.

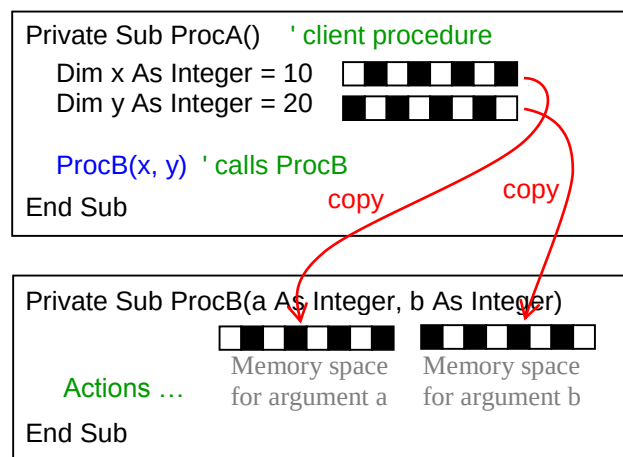


Figure 7 Passing arguments to parameters by value in procedure call

In Figure 7, ProcA() calls ProcB() and passes two arguments x (carrying value 10) and y (carrying value 20) to ProcB(). In this process of procedure call, the value of variable x (10) in ProcA() is copied to parameter a of ProcB(), and the value of variable y (20) in ProcA() is copied to parameter b of ProcB(). Because arguments x and y in ProcA() and parameters a and b of ProcB() are independent memory spaces in this by-value argument passing, no matter how values in a and b are changed by ProcB during execution, values in variables x and y in ProcA are not affected. This is demonstrated by Example 4 below.



Example 4: Create a new project and add one button (btnTest) in the main form. Then add the WeightedSum sub procedure into the form class and implement the button Click event procedure.

```

Private Sub btnTest_Click(...) Handles btnTest.Click
    Dim x As Integer = 10

```

```

Dim y As Integer = 20
WeightedSum(x, y) ' calls WeightedSum, pass a to x, pass b to y

' after executing sub WeightedSum, values in a and b are not changed
MessageBox.Show("x = " & x)
MessageBox.Show("y = " & y)
End Sub

Private Sub WeightedSum(ByVal a As Integer, ByVal b As Integer)
    Dim c As Integer
    a *= 0.4 ' value in variable a is changed, same as a = a * 0.4
    b *= 0.6 ' value in variable b is changed, same as b = b * 0.6
    c = a + b
    MessageBox.Show("a = " & a)
    MessageBox.Show("b = " & b)
End Sub

```

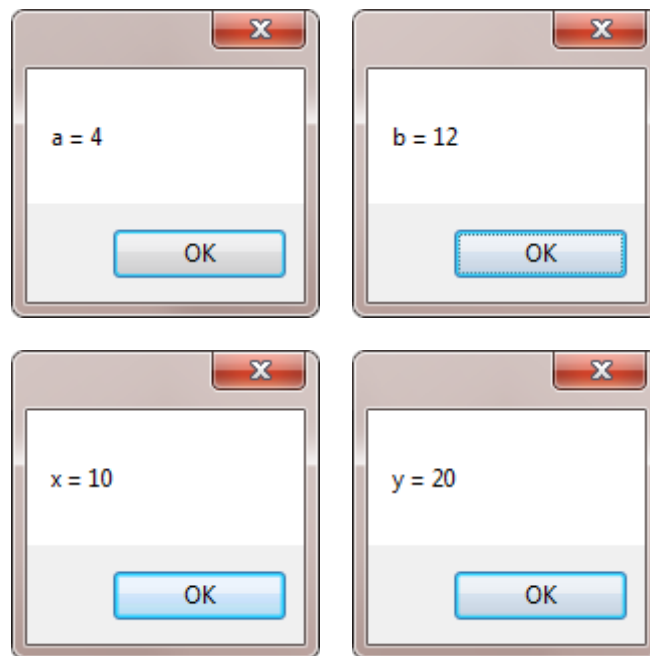


Figure 8 Running results of weighted sum program.
(With ByVal argument passing, original values in the client are not changed)

In this example, the button clicking event procedure declares two variables *x* and *y* and assigns values 10 and 20 to them respectively. The `WeightedSum()` sub procedure has two parameters *a* and *b*, both explicitly declared with keyword `ByVal`. When the event procedure (client) calls sub procedure `WeightedSum()`, it passes the value of *x* to parameter *a* and the value of *y* to parameter *b*. When sub procedure `WeightedSum()` is executed, values in variable *a* and *b* are changed and each is reported by a message boxes. However, values of *x* and *y* in the client procedure are unchanged as reported by another two message boxes in the event procedure (Figure 8).

5.2 ByVal

In a procedure call, client arguments can be passed to parameters of a procedure by reference (`ByRef`) when necessary. This is achieved by using keyword `ByRef` in

parameter declaration in the procedure header. When a parameter is declared with keyword `ByRef`, no memory space is allocated to the parameter to hold data for the procedure. Instead, only a small piece of memory space is set aside for the parameter to hold an address of the incoming data (Figure 9). Therefore, when the procedure is called, no data value is copied into a `ByRef` parameter. When the procedure being called is executed, if the parameter value is updated in computation, the update is made to the source argument in the client procedure.

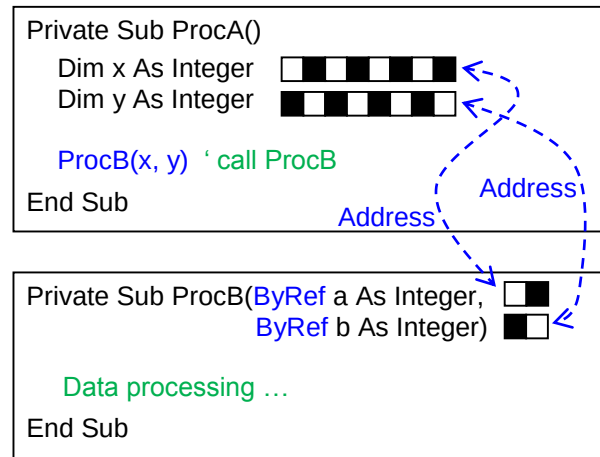


Figure 9 Passing parameters by reference

In Figure 9, procedure `ProcA()` calls `ProcB()` and passes two arguments `x` and `y` to it. Instead of copying the values of arguments `x` and `y` to parameters `a` and `b`, it only passes the addresses of the arguments to the parameters because parameters `a` and `b` are declared with the `ByRef` keyword. As a result, parameter `a` holds the address of `x` and parameter `b` holds the address of `y`. Therefore, parameters `a` and `b` can be regarded as proxies of `x` and `y` respectively. During data processing in `ProcB()`, if the value of `a` and `b` are updated, the updates are in fact made to variable `x` and `y` in `ProcA()`. When `ProcB()` finishes execution and variables `a` and `b` are deleted, only the small amount of memory used to hold the copy of addresses are recycled.

A good analogy for argument passing in procedure calls is file transmission between two people by email. `ByVal` argument passing is similar to attaching a file to the email. When the recipient receives the email, he/she receives a copy of the file. No matter what changes are made to the file by the recipient, the original document at the sender is not changed. On the other hand, `ByRef` argument passing is like to send a link to the file at a writeable shared location on a network. The recipient does not receive a copy of the file, but instead an addresses of it. If the recipient makes changes to the file at the shared location, the sender also sees the updates.



Example 5: To demonstrate parameter passing by reference, let's make a little change to the `WeightedSum` sub procedure in Example 4 by replacing keyword `ByVal` with `ByRef`.

```
Private Sub WeightedSum(ByRef a As Integer, ByRef b As Integer)
    Dim c As Integer
    a *= 0.4 ' value in variable a is changed
    b *= 0.6 ' value in variable b is changed
    c = a + b
```

```

    MessageBox.Show("a = " & a)
    MessageBox.Show("b = " & b)
End Sub

```

Keep the button click event unchanged and run the program. You will see the following results:

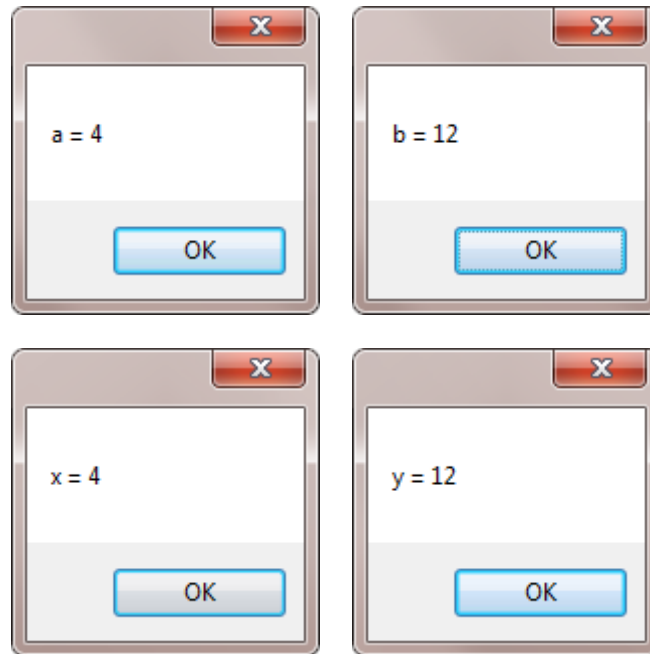


Figure 10 Running results of the weighted sum program
(With ByRef argument passing by reference, the original values of variables are changed)

Since ByRef can cause the original value of variables in the client procedure to change, the keyword must be used with care. However, passing arguments by reference can be very useful in certain situations. For example, we can use it to create a sub procedure to swap values between two variables.



Example 6: Create a VB project named “SwapNumbers” in which the main form contains one button (btnSwap). Then enter the Swap sub procedure in the form class and implement the button click event procedure as the following.

```

Private Sub Swap (ByRef a As Integer, ByRef b As Integer)
    Dim tmp As Integer
    tmp = a    ' uses a variable to temporarily hold the value of variable a
    a = b      ' pushes the value of variable b to variable a
    b = tmp    ' pushes the value of variable tmp to variable b
End Sub

Private Sub btnSwap_Click(...) Handles btnSwap.Click
    Dim x As Integer = 10
    Dim y As Integer = 20

    ' display values of x and y
    MessageBox.Show("x = " & x & Chr(13) & _

```

```

        "y = " & y, "Before Swapping")

Swap(x, y) ' calls sub Swap, passes x to a, y to b

' after executing Swap, display x and y again
MessageBox.Show("x = " & x & Chr(13) & _
        "y = " & y, "After Swapping")
End Sub

```

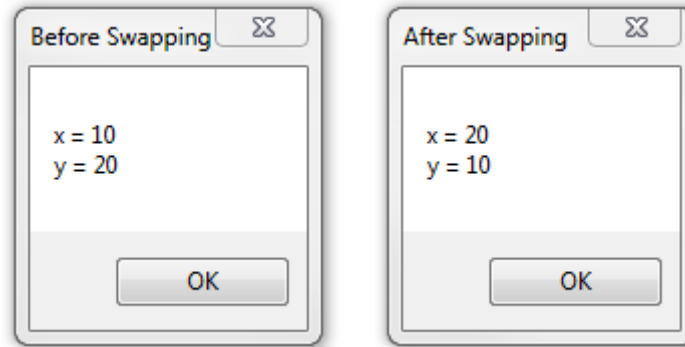


Figure 10 Execution result of the value swapping project

The Swap function is very useful in sorting numbers and strings which will be introduced in Chapter 6.

Summary and Highlights

A procedure is a self-contained unit of code to carry out certain tasks. There are two types of procedures in VB: function and sub procedure. A function usually takes some input data as parameters, process the data, and returns a result. A sub procedure can also take some input data and carry out some tasks. But a sub procedure does not return a result like a function does. The order of procedures in a class does not matter, but procedures cannot be nested or chained. Using procedures is an important way to decompose large jobs into small modules in procedural programming.

A procedure can be called by another which is referred to as a client. If a procedure declares parameters, the client must provide the exact number and type of arguments (values) in the right order when calling the procedure. While calling a procedure, arguments provided by the client are passed to corresponding parameters of the client. This process is called argument passing.

Argument passing in VB can occur in two different ways, by value (ByVal) or by reference (ByRef). If not explicitly specified in the parameter list of a procedure, the default way is ByVal in which argument values are copied to corresponding parameters. With ByVal argument passing, whatever changes are made to the parameters by a procedure being called, the original values of the argument in the client are not changed. In ByRef parameter passing, only the addresses of the arguments are passed to corresponding parameters. Therefore, when the procedure being called changes the parameters' values, changes are actually made to the arguments in the client procedure. If the ByRef keyword

is properly used, it can be very helpful and can achieve fantastic effect. But be careful in using this keyword.

In programming with procedures and flow control blocks, variables declared in different locations can have different scopes. Generally, variables declared directly under a class block is available to all procedures in the class, and these variables are called class-level variables. Variables, including parameters, declared directly under a procedure is available to all code below the declaration code line. Variables declared inside an If block is only available to the block where a condition is true. Variables declared inside a repetition block is only available to that repetition block.

Exercises

1. Answer the following questions.
 - 1) Why do we need to use procedures in programming?
 - 2) What is a function?
 - 3) What is a sub procedure?
 - 4) What are differences between functions and procedures?
 - 5) Why sub procedures do not have a data type?
 - 6) What is the structure of a function?
 - 7) What are arguments (parameters)?
 - 8) How do you declare arguments of procedures?
 - 9) How do you call a function?
 - 10) How do you pass parameters from one procedure to another?
 - 11) How does the program flow in a procedure in which other procedures are called?
 - 12) What is the scope of a variable?
 - 13) Can one procedure see variables declared in another procedure?
 - 14) Can we declare variable of same name in different procedures?
 - 15) Can two procedures in one class share common variables?
 - 16) When does a variable declared in a procedure die?
 - 17) What does keyword ByVal do?
 - 18) What does keyword ByRef do?
 - 19) What happens when passing parameters by value in a function call?
 - 20) What happens when passing parameters by reference in a function call?

2. Analyze the following code and determine program outputs.

1)

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim a As Double, b As Double, h As Double
    b = 25
    h = 20
    a = TriangleArea(b, h)
    lstOutput.Items.Add("Area of triangle 1 is " & CStr(a))
    b = 23
    h = 15
    a = TriangleArea(b, h)
    lstOutput.Items.Add("Area of triangle 2 is " & CStr(a))
End Sub

Private Function TriangleArea(base As Double, height As Double) As Double
    Dim area As Double = base * height / 2
    Return area
End Function
```

2) Suppose you enter "75" in the text box

```
Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim pages As Integer = CInt(txtPages.Text)
    Dim cost As Single = PrintingCost(pages)
    lstOutput.Items.Add("Your printing cost is " & FormatCurrency(cost))
End Sub

Private Function PrintingCost(pages As Integer) As Single
    Dim cost As Single
    If pages <= 10 Then
```

```

        cost = pages * 0.1
    ElseIf pages <= 50 Then
        cost = 1 + (pages - 10) * 0.08
    Else
        cost = 4.2 + (pages - 50) * 0.05
    End If
    Return cost
End Function

```

3)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim name As String, score As Integer
    name = "John"
    score = 85
    lstOutput.Items.Add(name & " receives '" & Grade(score) & "'")
    name = "Susan"
    score = 91
    lstOutput.Items.Add(name & " receives '" & Grade(score) & "'")
End Sub

```

```

Private Function Grade(score As Integer) As String
    Select Case score
        Case Is >= 90
            Return "A"
        Case Is >= 80
            Return "B"
        Case Is >= 70
            Return "C"
        Case Is >= 60
            Return "D"
        Case Else
            Return "F"
    End Select
End Function

```

4)

```

Private Function CanVote(age As Integer) As Boolean
    If age >= 18 Then
        Return True
    Else
        Return False
    End If
End Function

```

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim name As String, age As Integer
    name = "Marry"
    age = 16
    If CanVote(age) Then
        lstOutput.Items.Add(name & " can vote.")
    Else
        lstOutput.Items.Add(name & " cannot vote.")
    End If
End Sub

```

5) Suppose you enter “Barack” in the name text box

```

Private Function GetFirstName(fullName As String) As String
    Dim x As Integer = fullName.IndexOf(" ")
    If x < 0 Then
        Return Nothing
    Else
        Return fullName.Substring(0, x)
    End If
End Function

```

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim name As String = txtName.Text
    Dim fname As String = GetFirstName(name)
    If fname Is Nothing Then
        lstOutput.Items.Add("You provided a wrong full name")
    Else
        lstOutput.Items.Add("First name is: " & fname)
    End If
End Sub

```

6)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim name As String, score As Integer
    name = "Adam"
    score = 95
    PrintResult(name, Grade(score))
    name = "Bryce"
    score = 78
    PrintResult(name, Grade(score))
End Sub

```

```

Private Function Grade(score As Integer) As String
    Select Case score
        Case Is >= 90
            Return "A"
        Case Is >= 80
            Return "B"
        Case Is >= 70
            Return "C"
        Case Is >= 60
            Return "D"
        Case Else
            Return "F"
    End Select
End Function

```

```

Private Sub PrintResult(name As String, grade As String)
    lstOutput.Items.Add(name & ": " & grade)
End Sub

```

7)

```

Private Sub btnStart_Click(...) Handles btnStart.Click
    Dim r As Double = 10
    Dim a As Double = 0
    Dim p As Double = 0
    ComputeCircle(r, a, p)
    lstOutput.Items.Add("Circle area = " & Math.Round(a, 2))
    lstOutput.Items.Add("Circle perimeter = " & Math.Round(p, 2))

```

```
End Sub
```

```
Private Sub ComputeCircle(ByVal radius As Double,  
                           ByRef Area As Double,  
                           ByRef Perimeter As Double)  
    Area = Math.PI * radius * radius  
    Perimeter = Math.PI * radius * 2  
End Sub
```

8)

```
Private Sub btnStart_Click(...) Handles btnStart.Click  
    Dim str1 As String = "Good"  
    Dim str2 As String = "Morning"  
    SwapWords(str1, str2)  
    lstOutput.Items.Add(str1 & " " & str2)  
End Sub
```

```
Private Sub SwapWords(a As String, b As String)  
    Dim tmp As String = a  
    a = b  
    b = tmp  
End Sub
```

9)

```
Private Sub btnStart_Click(...) Handles btnStart.Click  
    Dim a As Double = 10  
    Dim b As Double = 20  
    lstOutput.Items.Add("a = " & CStr(a) & ", b = " & CStr(b))  
  
    Dim r As Double = Ratio(a, b)  
    lstOutput.Items.Add("a = " & CStr(a) & ", b = " & CStr(b))  
    lstOutput.Items.Add("ratio = " & Math.Round(r, 4))  
End Sub
```

```
Private Function Ratio(ByVal x As Double, ByRef y As Double) As Double  
    y = x + y  
    Return x / y  
End Function
```

10) Suppose you enter “George W. Bush” in the text box (txtName).

```
Private Sub btnStart_Click(...) Handles btnStart.Click  
    Dim fname As String = ""  
    Dim lname As String = ""  
    GetName(fname, lname)  
    If fname.Length > 0 And lname.Length > 0 Then  
        PrintName(fname, lname)  
    Else  
        MessageBox.Show("Wrong full name entered.", "Error",  
                        MessageBoxButtons.OK, MessageBoxIcon.Error)  
    End If  
End Sub  
  
Private Sub GetName(ByRef FirstName As String, ByRef LastName As String)  
    Dim name As String = txtName.Text  
    Dim x1 As Integer = name.IndexOf(" ")  
    Dim x2 As Integer = name.LastIndexOf(" ")
```

```

    If x1 > 0 And x2 > 0 Then
        FirstName = name.Substring(0, x1)
        LastName = name.Substring(x2 + 1)
    End If
End Sub

Private Sub PrintName(FirstName As String, LastName As String)
    lstOutput.Items.Add("First Name: " & FirstName)
    lstOutput.Items.Add("Last Name: " & LastName)
End Sub

```

3. Create a new VB project named “Euclid” to compute the Euclidean distance between two locations. The project must use a function (name it EucDist()) to carry out distance computation. The function must use four Double type parameters to receive four coordinate values provided by the client, and return a Double type result. You can follow Example 2 to design an application GUI.
4. Create a new VB project named “NDVI” for computing Normalized Difference Vegetable Index (NDVI). $NDVI = \frac{(NIR - VIS)}{(NIR + VIS)}$ where NIR stands for near-infrared value and VIS stands for visible (red) value. The application GUI should provide two text boxes for the user to enter the NIR and VIS values, a button to start compute, and a text box to display result. Your program must use a function (name it NDVI()) to compute the NDVI index. It should also include a function (name it ValidateInput()) to check if necessary values are entered into the text boxes.
5. Create a new VB project named “PopGrowth” to compute population growth. The program uses a GUI for the user to enter a city name, current population, annual population growth rate, and number of years. When a compute button is clicked, the program calls a function named NewPopulation() to compute new population. Function NewPopulation() must have three parameters: current population (Long), annual growth rate (Double), and number of years (Integer), and it must return a Long type result. Refer to the following screenshot to design your application GUI.

Tip: given a population *pop*, the population grows by *pop * rate* each year. Therefore, the population of the next year is

$pop += pop * rate$

This is the same as:

$pop = pop + pop * rate$

Or

$pop = pop * (1 + rate)$

Note: The rate is entered in percentage, it must be divided by 100 in computation.